StudySync AI: An Intelligent Study Planner Using Constraint Satisfaction

A Classical AI Approach Based on Backtracking Search and the Minimum Remaining Values Heuristic

Naren Kartic B.
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
narenkartic.b.2024.csecs@rajalakshmi.edu.in

Abstract—StudySync AI assists students in organizing their academic activities by considering subject requirements, examination dates, available study hours, subject priorities, and individual scheduling preferences. The system models study planning as a Constraint Satisfaction Problem (CSP), in which study sessions are variables, available time slots form their domains, and academic requirements define the constraints. Hard constraints, such as avoiding overlapping sessions and respecting available study hours, ensure schedule feasibility, while soft preferences, such as prioritizing difficult subjects and allocating sufficient revision time, improve the usefulness of the generated plan. Classical AI search techniques, namely backtracking search and the Minimum Remaining Values (MRV) heuristic, explore possible schedules efficiently, while a constraint validation mechanism identifies conflicts and eliminates assignments that violate requirements. The system follows a structured workflow of understanding student inputs, defining constraints, searching for feasible assignments, and validating the resulting schedule. It is developed using Python and Streamlit, providing an interactive and user-friendly web interface, and uses no machine learning or deep learning. Experiments on a reference implementation show that MRV with forward checking reduces search effort from tens of thousands of nodes to tens of nodes on a moderately constrained scenario and correctly detects infeasible requirements, demonstrating the practical application of foundational AI concepts to everyday academic challenges.

Keywords—study planning, constraint satisfaction problem, backtracking search, minimum remaining values, forward checking, scheduling, classical artificial intelligence

I. INTRODUCTION

Students at every level of education must balance several subjects, each with its own syllabus, difficulty, and examination date, while working within a limited number of available hours each day. Planning this workload is rarely trivial. A student who studies only the subjects they enjoy risks neglecting those that are more demanding; one who spreads time evenly across all subjects may leave too little time for the hardest ones; and one who plans without regard to examination dates may find that the most urgent paper has received the least preparation. Most students handle this by writing timetables manually, and such timetables are frequently abandoned the moment an unexpected commitment appears.

The study planning task has a natural logical structure. A plan consists of a set of study sessions, each of which must be placed in some time slot, and the placement must obey rules: two sessions cannot occupy the same slot, no session may be placed in a period during which the student is unavailable, and every subject must receive enough sessions before its examination. Problems of exactly this form—a set of variables, a finite set of values for each, and a set of restrictions among them—are the subject of the Constraint Satisfaction Problem (CSP) formalism, one of the foundational topics of classical Artificial Intelligence [1], [4].

StudySync AI is a study-planning assistant built on this observation. It accepts the subjects a student must prepare, the examination date of each subject, the hours the student has available, the relative priority of each subject, and a small set of scheduling preferences. It then models the planning problem as a CSP in which study sessions are the variables, the available time slots form their domains, and academic requirements define the constraints. Hard constraints, such as the prohibition of overlapping sessions, must always be satisfied; soft preferences, such as scheduling difficult subjects earlier in the day or reserving revision time before an examination, are used to improve the quality of the plan.

To search for a valid schedule, the system employs classical backtracking search combined with the Minimum Remaining Values (MRV) heuristic, which always selects the session that currently has the fewest legal placements. A constraint validation mechanism detects conflicts during the search and eliminates assignments that would violate the requirements. The system follows a structured workflow consisting of understanding student inputs, defining constraints, searching for feasible assignments, and validating the resulting schedule.

StudySync AI is implemented in Python with a Streamlit web interface. It deliberately uses no machine learning or deep learning: every decision the system makes can be traced to an explicit constraint or search step, which makes the generated plan explainable and the project a direct application of the techniques taught in Foundations of Artificial Intelligence (FOAI). The remainder of this paper is organized as follows. Section II states the problem and motivation, Section III sets out the objectives, Section IV reviews the literature, Section V contrasts existing and proposed systems, Section VI describes the architecture, Sections VII and VIII present the CSP formulation and algorithms, Section IX describes the implementation, Section X reports results, Section XI describes testing, and Sections

XII–XIV discuss limitations, learning outcomes, and conclusions.

II. PROBLEM STATEMENT AND MOTIVATION

A. Problem Statement

Given a set of subjects, each with a required amount of preparation, a priority, and an examination date, together with the daily time slots in which the student is available, produce a conflict-free study schedule that completes the required preparation before each examination while respecting the student's preferences as far as possible.

The difficulty of this problem is combinatorial. Even a modest plan of fourteen one-hour sessions across seven days with four slots per day offers twenty-eight possible slots for each session, giving a search space that grows exponentially with the number of sessions. Checking every combination by hand is infeasible, and ad-hoc manual planning offers no guarantee that a conflict-free arrangement has been found, or that one exists at all. When a constraint changes—a new commitment appears, an examination is rescheduled—the plan must be rebuilt, and manual re-planning is both slow and error-prone.

B. Motivation

The first motivation is practical. Disorganized preparation is a common cause of avoidable stress and under-performance, and an automated planner that removes the mechanical burden of arranging sessions allows students to spend their effort on studying rather than on scheduling.

The second motivation is pedagogical. Study planning is an everyday problem that maps cleanly onto the CSP model, so it provides a concrete and relatable setting in which to apply backtracking, constraint propagation, and variable-ordering heuristics. The relationship between the real-world requirement (for example, "do not study two subjects at the same time") and its formal counterpart (an all-different constraint on slot assignments) is immediately visible, which makes the project a useful demonstration of how abstract AI concepts solve ordinary problems.

The third motivation is explainability. Learners are more likely to follow a schedule they understand. Because the system's reasoning consists of explicit constraints and a search process whose steps can be displayed, StudySync AI can show why a session was placed where it was, or why no feasible schedule exists, in terms a student can follow. This is difficult to achieve with opaque statistical models.

III. OBJECTIVES

The main objective of StudySync AI is to assist students in organizing their academic activities by considering subject requirements, examination dates, available study hours, subject priorities, and individual scheduling preferences. The specific objectives are:

- To model the study planning problem as a Constraint Satisfaction Problem with sessions as variables, time slots as domains, and academic requirements as constraints.
- To accept student inputs describing subjects, examination dates, available hours, priorities, and preferences through an interactive interface.
- To define hard constraints that guarantee schedule feasibility, such as non-overlapping sessions and adherence to available study hours.
- To define soft preferences that improve plan quality, such as prioritizing difficult subjects and allocating sufficient revision time.
- To implement backtracking search for exploring candidate study schedules systematically.
- To apply the Minimum Remaining Values (MRV) heuristic to reduce the number of search nodes explored.
- To implement a constraint validation mechanism that identifies conflicts and eliminates assignments that violate requirements.
- To report clearly when no feasible schedule exists, and to identify which requirements are in conflict.
- To present student inputs, constraints, the AI decision-making process, and the generated plan clearly in a visually appealing Streamlit interface.
- To reduce the repetitive manual adjustment of timetables when requirements change.
- To promote balanced preparation across subjects and help students develop consistent study habits.
- To demonstrate the practical application of foundational AI methods without machine learning or deep learning.

IV. LITERATURE SURVEY

A. Constraint Satisfaction Foundations

The CSP formalism represents a problem by a set of variables, a finite domain of values for each variable, and a set of constraints restricting which combinations of values are permitted [4], [6]. Mackworth's early work on consistency in networks of relations introduced node, arc, and path consistency as techniques for removing values that cannot participate in any solution, thereby pruning the search space before and during search [2]. Surveys by Kumar [5] and the comprehensive treatments by Dechter [4] and Tsang [6] consolidate these ideas and describe how consistency enforcement, backtracking, and heuristics interact. Russell and Norvig present CSPs as a core topic of introductory AI, using map colouring, scheduling, and cryptarithmetic as running examples [1].

B. Backtracking Search and Variable Ordering

Backtracking is the standard systematic search procedure for CSPs. Golomb and Baumert formalized backtrack programming as a general method for exploring a tree of partial assignments and abandoning any branch that violates a constraint [11]. Bitner and Reingold later described

practical techniques for improving backtrack programs, including the ordering of choices and the early detection of dead ends [10]. Haralick and Elliott showed that ordering variables so that the most constrained one is chosen first—the "fail-first" principle, later popularized as the Minimum Remaining Values heuristic—can substantially reduce the size of the search tree, and they introduced forward checking, which removes values from the domains of unassigned variables as soon as an assignment makes them inconsistent [3]. These two ideas, MRV and forward checking, form the basis of the search strategy used in this project.

C. Timetabling and Scheduling Applications

Scheduling and timetabling are among the most widely studied applications of constraint-based methods. Schaerf's survey of automated timetabling classifies school, university, and examination timetabling problems and compares the techniques applied to them, noting the central distinction between constraints that must hold and those that are merely desirable [8]. Burke and Petrovic review later research directions in automated timetabling, including multi-criteria optimization and the handling of preferences [9]. Brailsford, Potts, and Smith survey CSP algorithms and applications, with particular emphasis on scheduling, resource allocation, and timetabling [12]. The Handbook of Constraint Programming collects both the theory and the applied practice of this area [7]. The hard/soft constraint distinction found in this literature directly informs the formulation used in StudySync AI.

D. Learning Science Considerations

A study planner is only useful if its schedules reflect how people learn effectively. Reviews of the distributed-practice literature report that spacing study sessions over time produces better retention than massing the same amount of study into a single block [13]. A broader review of learning techniques rates distributed practice and practice testing among the most effective strategies available to students [14]. These findings motivate the soft constraints of StudySync AI, which favour spreading a subject's sessions across several days and reserving revision sessions shortly before an examination rather than placing all preparation in a single day.

E. Research Gap

Existing timetabling research largely addresses institutional problems—course, classroom, and examination scheduling—that involve many resources and administrators. Consumer study-planning applications, on the other hand, commonly provide calendars and reminders but leave the actual arrangement of sessions to the user. Few present the planning step as an explicit, inspectable constraint-solving process. StudySync AI occupies this gap: it applies established CSP methods to the individual student's planning problem and exposes the constraints, the search steps, and the validation outcome to the user, using only classical AI techniques.

F. Summary of Surveyed Work

Table I consolidates the surveyed work and indicates how each contribution is used in this project.

TABLE I
SUMMARY OF LITERATURE

Ref.	Contribution	Use in StudySync AI
[1]	CSP as core AI topic; search and heuristics	Overall CSP framing of the project
[2]	Node, arc, and path consistency	Basis for domain pruning and future AC-3
[3]	Fail-first ordering; forward checking	MRV heuristic and forward checking
[4]-[6]	Comprehensive CSP treatments and survey	Formulation of variables, domains, constraints
[8], [9]	Automated timetabling surveys	Hard versus soft constraint distinction
[10], [11]	Backtrack programming techniques	Backtracking search engine
[12]	CSP algorithms and applications	Scheduling-oriented CSP modelling
[13], [14]	Distributed practice and effective study	Soft preferences S2 and S3

V. EXISTING SYSTEM AND PROPOSED SYSTEM

A. Existing System

Students currently plan their study through handwritten timetables, spreadsheets, generic calendar applications, or to-do list tools. These tools record a plan but do not construct one. The student decides which subject goes in which slot, checks for clashes by eye, and revises the plan manually whenever circumstances change.

B. Limitations of Existing Approaches

- Manual construction of schedules is time-consuming and must be repeated whenever requirements change.
- Overlapping sessions and over-allocated days are detected, if at all, only by inspection.
- Examination dates and subject priorities are not systematically balanced against available time.
- Difficult subjects frequently receive insufficient time, and revision is left to the last moment.
- No indication is given when a set of requirements is mutually impossible to satisfy.
- Generic calendar tools offer no explanation of why a plan is arranged as it is.

C. Proposed System

StudySync AI constructs the schedule automatically. It converts the student's inputs into a CSP, applies backtracking search with the MRV heuristic to find an assignment that satisfies all hard constraints, evaluates the soft preferences to select a good plan among those that are feasible, and validates the final schedule before presenting it. If the constraints

cannot be satisfied simultaneously, the system reports the conflict instead of producing an invalid plan.

D. Advantages of the Proposed System

- Guaranteed conflict-free schedules with respect to the declared hard constraints.
- Efficient search through MRV-guided backtracking.
- Priority-aware and exam-aware allocation of study time.
- Immediate re-planning when inputs change.
- Transparent, explainable reasoning that can be shown to the student.
- Interactive, visually clear presentation in a web interface.
- No training data or machine learning models required.

VI. SYSTEM ARCHITECTURE AND METHODOLOGY

The system follows a structured workflow in which each stage consumes the output of the previous stage, as shown in Fig. 1. The workflow consists of understanding student inputs, defining constraints, searching for feasible assignments, and validating the resulting schedule.

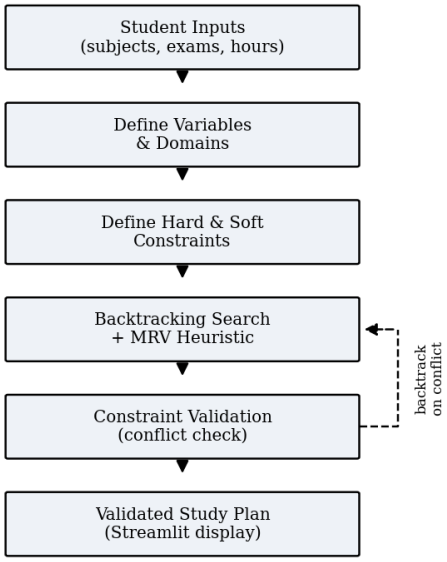


Fig. 1. Workflow of the StudySync AI planning pipeline.

A. Understanding Student Inputs

The student supplies the list of subjects and, for each subject, the examination date, the amount of preparation required (expressed as a number of study sessions), a priority or difficulty rating, and any subject-specific preference. The student also specifies the days and time slots during which they are available, together with global preferences such as the maximum study load per day. The input module validates these values—for example, rejecting an examination date in the past—before passing them on.

B. Defining Constraints

The constraint-definition stage converts the inputs into a CSP. One variable is created for every required study session; the domain of each variable is the set of available slots that fall before the examination of its subject; and the hard constraints and soft preferences of Section VII are instantiated. This stage also performs a quick feasibility pre-check: if the total number of sessions required exceeds the total number of available slots, the problem is immediately reported as infeasible.

C. Searching for Feasible Assignments

The search module runs backtracking search with MRV variable ordering and forward checking (Section VIII). Sessions are assigned to slots one at a time; after each assignment, the domains of the remaining sessions are filtered; and if any domain becomes empty, the search backtracks and tries a different value.

D. Validating the Schedule

When the search produces a complete assignment, the validation module re-checks every hard constraint independently of the search procedure. This separation provides a safeguard: even if a defect existed in the search code, an invalid schedule would not reach the student. The validated schedule is then scored against the soft preferences and displayed.

E. Presentation

The Streamlit interface presents the inputs, the derived constraints, a summary of the search process (nodes explored and backtracks), and the final plan in a calendar-style grid, so that the student can see not only the result but also how it was obtained.

F. Component Interaction

The components interact through a small number of well-defined data structures. The input module produces a list of subject records and an availability set. The model builder consumes these and produces the CSP: the list of variables, the dictionary of domains, and the constraint parameters. The search engine consumes the CSP and produces either a complete assignment or a failure together with search statistics. The validator consumes the assignment and the constraint parameters and produces a verdict and the soft-preference cost, and the interface consumes all of these to render the four views. Because each component communicates only through these data structures, any one of them—for example, the search engine—can be replaced or improved without affecting the others.

G. Why a Constraint-Based Approach

Several alternative techniques could be applied to scheduling. Table II compares them with the CSP approach on the criteria most relevant to a student-facing planner.

TABLE II
ALTERNATIVE APPROACHES TO STUDY SCHEDULING

Approach	Guarantees valid plan	Explainable	Needs training data
Greedy rules	No	Yes	No
Genetic algorithm	Not guaranteed	Limited	No
Neural / learned model	No	No	Yes
CSP with backtracking	Yes	Yes	No

A greedy rule, such as always scheduling the most urgent subject next, is simple and explainable but may paint itself into a corner and give no warning that it has done so. Evolutionary methods can optimize quality but do not guarantee that every hard constraint is satisfied, and their stochastic nature means that the same input may give different plans. Learned models would require data on successful study schedules that is not readily available and would not provide guarantees. A CSP formulation with complete search provides a guarantee that the output satisfies every hard constraint and a proof when none exists, while remaining transparent, which suits a tool whose output students are expected to follow.

VII. CONSTRAINT SATISFACTION FORMULATION

A CSP is defined by a triple (X, D, C), where X is a set of variables, D assigns each variable a finite domain of values, and C is a set of constraints over subsets of the variables [1], [4]. A solution is a complete assignment of values to variables that violates no constraint. This section defines each element of the triple for the study-planning problem.

A. Variables

Each required study session is a variable. If subject s requires n(s) sessions, the variables for that subject are s#1, s#2, …, s#n(s). The total number of variables is therefore the sum of n(s) over all subjects. For a student preparing five subjects with a combined requirement of fourteen sessions, the CSP has fourteen variables.

B. Domains

A time slot is a pair (d, t), where d is the day and t the slot index within that day. The domain of a variable is the set of slots that are (i) marked as available by the student and (ii) earlier than the examination date of the variable's subject. Domains therefore differ between variables: sessions of a subject whose examination is near have smaller domains than those of a subject whose examination is far away. This difference in domain size is precisely what the MRV heuristic exploits.

C. Hard Constraints

Hard constraints define feasibility. A schedule that violates any of them is rejected. They are summarized in Table III.

TABLE III
HARD CONSTRAINTS

ID	Constraint	Formal statement
H1	No overlap	$x(u) \neq x(v)$ for all $u \neq v$
H2	Availability	$x(v) \in A$ (student-available slots)
H3	Exam deadline	$day(x(v)) < exam(subject(v))$
H4	Daily load	$\lvert\{v : day(x(v)) = d\}\rvert \leq L$ for every day d
H5	Per-subject daily cap	sessions of one subject on a day $\leq m$
H6	Session order	x(s#i) precedes x(s#j) for $i < j$

Constraint H1 is an all-different constraint over the slot assignments and prevents the student from being scheduled for two sessions at once. H2 and H3 are unary constraints that are applied when the domains are built. H4 limits the total study hours in a day to a student-chosen maximum L, preventing over-loaded days. H5 prevents a single subject from monopolizing a day. H6 orders the sessions of each subject chronologically; this constraint does not change what the student receives, since sessions of a subject are interchangeable, but it eliminates symmetric solutions that differ only by swapping equivalent sessions, and thereby removes redundant branches from the search tree.

D. Soft Preferences

Soft preferences express what makes a feasible schedule a good one. They are not required to hold, but violations are penalized. The preferences used are summarized in Table IV.

TABLE IV
SOFT PREFERENCES AND PENALTIES

ID	Preference	Penalty applied when
S1	Difficult subjects earlier in the day	High-priority session in a late slot
S2	Spread sessions across days	Subject sessions clustered on few days
S3	Revision before exam	No session in the 2 days before exam
S4	Balanced daily workload	Large difference between busiest and lightest days
S5	Student slot preference	Session placed in a disliked slot

The quality of a feasible schedule S is measured by a weighted penalty:

$$\text{Cost}(S) = \Sigma_i \, w_i \cdot P_i(S) \quad (1)$$

where $P_i(S)$ counts violations of preference S_i and w_i is its weight. The weights are chosen so that subject priority influences S1 and S3 more strongly for high-priority subjects. A lower cost indicates a more useful plan.

E. Problem Size and Difficulty

A CSP with n variables, each having a domain of size at most d, has up to d^n complete assignments. For the small scenarios in this paper, n lies between 14 and 27 and d between 8 and 20, so exhaustive enumeration is impractical, although systematic backtracking with pruning remains

tractable. Since CSP solving is NP-complete in general [4], the objective is not to avoid exponential worst-case behaviour but to reduce the effort required on realistic instances, which is what variable-ordering heuristics and constraint propagation are designed to do.

VIII. SEARCH ALGORITHMS

A. Backtracking Search

Backtracking search builds a schedule incrementally. It selects an unassigned variable, tries the values in its domain one by one, and for each value checks consistency with the assignments already made. If the value is consistent, it is assigned and the search recurses on the remaining variables. If no value succeeds, the procedure returns failure to its caller, which then tries its next value—this is the "backtrack" [10], [11]. The procedure is complete: if a schedule satisfying all hard constraints exists, it will be found, and if none exists, the search terminates after exhausting the tree and reports failure.

```
function BACKTRACK(assignment, unassigned):
  if unassigned is empty: return assignment
  v <- SELECT(unassigned)          # MRV
  for each value in ORDER(D[v]):
    if CONSISTENT(v, value, assignment):
      assignment[v] <- value
      saved <- FORWARD_CHECK(v, value)
      if no domain became empty:
        r <- BACKTRACK(assignment,
                       unassigned - {v})
        if r != failure: return r
      RESTORE(saved)
      remove v from assignment
  return failure
```

Algorithm 1: Backtracking search with MRV and forward checking.

B. Minimum Remaining Values Heuristic

In plain backtracking, the order in which variables are considered is fixed. MRV replaces this with a dynamic choice: at each step it selects the unassigned variable whose current domain is smallest:

$$v^* = \text{argmin over unassigned } v \text{ of } |D(v)| \quad (2)$$

The reasoning is that a variable with few remaining options is the most likely to run out of them. Selecting it early causes an inevitable failure to be discovered near the root of the search tree, where it prunes a large subtree, instead of deep in the tree where much work has already been wasted. Ties are broken deterministically, so the system produces the same schedule for the same input. In the study-planning setting, MRV naturally schedules the sessions of subjects with imminent examinations or heavily restricted availability first, which matches how a human planner would proceed.

C. Forward Checking and Conflict Detection

After each assignment, forward checking removes from the domains of all unassigned variables any value that is now inconsistent with the assignment [3]. For example, once a session is placed in the slot (Day 2, Slot 3), that slot is removed from every other session's domain (constraint H1), and if the day has reached its daily load limit L, every slot on that day is removed (constraint H4). If any domain becomes empty, the current partial assignment cannot be completed, so the search backtracks immediately without descending further. This mechanism is the constraint validation step that identifies conflicts and eliminates violating assignments during planning.

D. Final Validation and Soft-Preference Evaluation

Once a complete assignment is found, an independent validator re-checks H1–H6 over the full schedule and computes Cost(S) from Eq. (1). Validation is deliberately implemented separately from the search so that correctness does not depend on the search code. If the student requests alternative plans, the search can continue past the first solution, and the lowest-cost valid schedule is retained.

E. Complexity

In the worst case, backtracking explores $O(d^n)$ nodes. MRV and forward checking do not change this worst-case bound but reduce the practical number of nodes, sometimes by orders of magnitude, as Section X shows. Each consistency check costs O(n) in the number of assigned sessions in the implementation described here, which is adequate for the instance sizes of a single student's study plan.

F. Worked Example

A small example illustrates how MRV and forward checking cooperate. Consider three subjects over three days with two slots per day, where Day 1 Slot 2 is blocked. Mathematics needs two sessions and its examination falls on Day 3, so its sessions may use only Days 1 and 2; Physics needs two sessions and English one, and both examinations fall after Day 3. The domains are therefore of size 3 for each Mathematics session and size 5 for each Physics and English session. Table V traces the search.

TABLE V

MRV TRACE FOR THE WORKED EXAMPLE

Step	Variable chosen (MRV)	Slot assigned	Domain sizes afterwards
1	Math#1 (size 3)	Day 1, Slot 1	Math#2 = 2; Phys = 4, 4; Eng = 4
2	Math#2 (size 2)	Day 2, Slot 1	Phys = 3, 3; Eng = 3
3	English (size 3)	Day 2, Slot 2	Phys = 2, 2
4	Physics#1 (size 2)	Day 3, Slot 1	Phys#2 = 1
5	Physics#2 (size 1)	Day 3, Slot 2	-

The two Mathematics sessions are selected first because their domains are the smallest, as the near examination removes Day 3 from their options. After each assignment, forward checking removes the used slot from every other domain and the domain sizes shrink, as shown in the final column. At step 3 English and Physics tie in domain size, and the deterministic tie-break by name selects English. By step 4 the remaining Physics sessions have only the two Day 3 slots left, and the ordering constraint H6 forces Physics#1 into the earlier of them. The search completes in five nodes

without backtracking. Had the search processed Physics first and placed it on Day 2, the Mathematics sessions would have lost one of their three options, and a later conflict could have forced backtracking; MRV avoids this by dealing with the scarce resource first.

G. Validation Procedure

The independent validator that follows the search is a direct translation of the hard constraints into checks over the finished schedule, as outlined in Algorithm 2. Its running time is quadratic in the number of sessions because of the pairwise overlap test, which is negligible for plans of this size.

```
function VALIDATE(schedule, params):
  for each pair (u, v) in schedule:
    if slot(u) == slot(v): return INVALID (H1)
  for each session v in schedule:
    if slot(v) not in params.available:
      return INVALID (H2)
    if day(v) >= params.exam[subject(v)]:
      return INVALID (H3)
  for each day d:
    if load(d) > params.L: return INVALID (H4)
    if per_subject_load(d) > params.m:
      return INVALID (H5)
  for each subject s:
    if sessions of s not in order:
      return INVALID (H6)
  return VALID, COST(schedule)
```

Algorithm 2: Independent validation of a completed schedule.

IX. IMPLEMENTATION

A. Technology Stack

StudySync AI is developed in Python, which provides concise data structures for representing domains and assignments, and Streamlit [15], which turns Python scripts into interactive web applications without separate front-end code. The use of Streamlit gives the project an interactive, visually appealing, and user-friendly interface while keeping the entire code base in a single language.

B. Modular Design

The code is organized into modules with clear responsibilities, as summarized in Table VI. This modular design supports a clear presentation of student inputs, scheduling constraints, the AI decision-making process, and the generated study plan, and allows each module to be tested independently.

TABLE VI
SOFTWARE MODULES

Module	Responsibility
Input module	Collects and validates subjects, exam dates, hours, priorities, preferences
Model builder	Creates variables, domains, and constraints from the inputs
Search engine	Backtracking with MRV and forward checking
Validator	Independent re-check of hard constraints; soft-cost scoring
Explainer	Records search statistics and conflict reasons for display
UI layer	Streamlit pages for inputs, constraints, process, and plan

C. Inputs

The input form uses standard Streamlit controls: text fields for subject names, date pickers for examination dates, sliders for priority and difficulty, numeric fields for the number of sessions, and a grid of check-boxes for slot availability. Using constrained controls instead of free text reduces invalid input and keeps the model builder simple.

D. Interface Presentation

The interface is divided into four views that mirror the workflow: Inputs, Constraints, AI Process, and Study Plan. The Constraints view lists the hard and soft constraints derived from the inputs in readable form. The AI Process view shows the number of variables, the domain sizes, the nodes explored, and the number of backtracks, as well as any conflict detected. The Study Plan view shows the generated schedule as a day-by-slot grid, with each subject shown in a distinct colour so the balance of preparation can be seen at a glance.

E. Handling Infeasible Inputs

When the search terminates without a solution, the system does not present an empty or partial plan. Instead it reports that the requirements cannot all be met and identifies the cause where possible—for example, that the sessions required before a particular examination exceed the slots available before it. The student can then relax a requirement, such as adding availability or reducing the number of sessions, and re-run the planner. This behaviour distinguishes a constraint-based planner from a heuristic that silently produces an overloaded timetable.

F. Data Representation

Internally, a subject is represented as a record containing its name, examination date, number of required sessions, and priority. A time slot is represented as a (day, slot) pair, and the set of available slots is stored as a set so that membership tests are fast. The CSP itself is stored as a dictionary mapping each variable to the list of values currently remaining in its domain, together with a second dictionary holding the current partial assignment. Forward checking records the domains it modifies so that they can be restored exactly when the search backtracks. Keeping the model, the assignment, and the search statistics in separate structures makes the search engine independent of the interface and easy to test in isolation.

G. Typical User Workflow

A typical session proceeds as follows. The student opens the application and enters the subjects they must prepare, choosing the examination date and priority for each. They mark the slots in which they are free, and optionally adjust

preferences such as the maximum number of hours per day. On pressing the plan button, the system builds the CSP, runs the search, validates the result, and switches to the plan view. If the student then discovers a new commitment, they mark the affected slots as unavailable and press the button again; the plan is regenerated within a fraction of a second, which removes the repetitive manual adjustment that is the central burden of conventional timetables.

H. Design Considerations

Three design decisions deserve mention. First, the search is deterministic: variables are ordered by domain size with ties broken by name, and values are tried in chronological order, so identical inputs always give identical plans, which makes the system testable and its behaviour predictable to the student. Second, hard and soft requirements are kept strictly separate: hard constraints can only prune the search, while soft preferences are used only to rank feasible solutions, so a preference can never cause an invalid schedule. Third, the system is designed to be extended through new constraints rather than new code paths: adding a requirement, such as a rest day before an examination, means adding one constraint to the model rather than modifying the search algorithm.

I. Software and Hardware Requirements

The system has modest requirements, since the scheduling problem is small and the interface runs in a browser:

- Software: Python 3, the Streamlit library, and a modern web browser.
- Hardware: any standard personal computer or laptop; no GPU or specialized hardware is needed because the system uses no machine learning.
- Operating system: platform-independent, since both Python and Streamlit run on Windows, Linux, and macOS.

X. RESULTS AND DISCUSSION

A. Experimental Setup

The search procedure described in Section VIII was evaluated on three synthetic study-planning scenarios of increasing difficulty, each with seven days and four one-hour slots per day. Scenario S1 is a lightly constrained plan, S2 is a moderately constrained plan with several blocked slots and near examination dates, and S3 is a deliberately over-constrained plan whose requirements cannot all be met. The scenarios are summarized in Table VII. For each scenario, three search configurations were compared: backtracking with a fixed variable order (static), backtracking with MRV, and backtracking with MRV and forward checking (MRV+FC). The measures reported are the number of search nodes explored, the number of backtracks, and elapsed time. Static search was given a cap of 30,000 nodes. The experiments use a reference implementation of the formulation in Section VII that applies constraints H1, H4, H5, and H6 in the search and H2 and H3 in the domains.

Scenario S2, used for the detailed illustrations below, comprises six subjects: Mathematics (four sessions, last study day Day 5), Physics (four sessions, Day 6), Chemistry (three sessions, Day 4), Computer Science (three sessions, Day 7), English (three sessions, Day 7), and Biology (three sessions, Day 6). Seven slots are blocked, representing the student's other commitments. At most two sessions of one subject may be placed on a day, and at most four sessions on any day.

Four measures are used to evaluate each run. The number of nodes explored counts every attempt to assign a value to a variable and is independent of hardware, making it the principal measure of search effort. The number of backtracks counts the times the search has to undo an assignment. Elapsed time is reported for completeness. Finally, the outcome records whether a valid schedule was found, proven impossible, or abandoned at the node cap. In every case in which a schedule was returned, the independent validator confirmed zero violations of the hard constraints.

TABLE VII
EXPERIMENTAL SCENARIOS

ID	Subjects	Sessions (variables)	Avg. domain size	Blocked slots
S1	5	14	20.4	5
S2	6	20	16.2	7
S3	6	27	8.4	7

B. Search Performance

TABLE VIII
SEARCH EFFORT BY CONFIGURATION

Scenario	Method	Nodes	Backtracks	Outcome
S1	Static	105	0	Solved
S1	MRV	105	0	Solved
S1	MRV + FC	14	0	Solved
S2	Static	30,130	1,746	Not solved (cap)
S2	MRV	441	11	Solved
S2	MRV + FC	31	4	Solved
S3	Static	160	32	No solution
S3	MRV	160	32	No solution
S3	MRV + FC	31	16	No solution

Table VIII and Fig. 2 summarize the results. On the lightly constrained scenario S1, all three configurations find a schedule without any backtracking; MRV makes no difference because almost every placement is legal, while forward checking reduces the nodes explored from 105 to 14, which is simply one node per session. The moderately constrained scenario S2 shows the benefit of the heuristic clearly. With a fixed variable order, the search explored the full cap of 30,130 nodes with 1,746 backtracks and had still not found a valid schedule, because it committed early to sessions with large domains and discovered conflicts involving the tightly constrained sessions only after exploring large subtrees. With MRV the same problem was

solved in 441 nodes, and with MRV and forward checking in 31 nodes with only four backtracks.

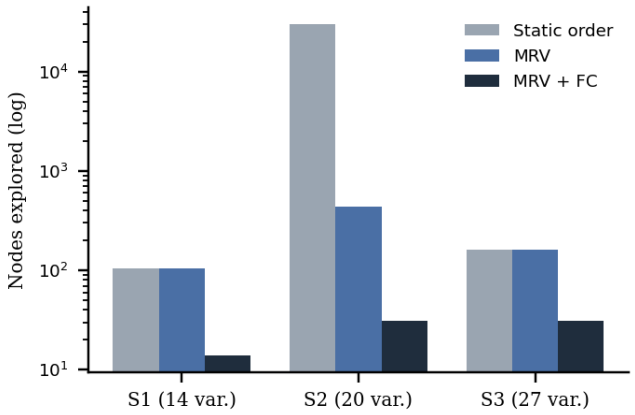


Fig. 2. Search nodes explored per scenario (logarithmic scale).

On the over-constrained scenario S3, all configurations correctly concluded that no valid schedule exists, because the backtracking search is complete. Forward checking established this after 31 nodes, compared with 160 for the other configurations. This outcome is itself a useful result for the student: instead of receiving a timetable that silently breaks a requirement, they are told that the requirements conflict and can relax them.

The elapsed times in this unoptimized prototype are all below a few milliseconds for the solved scenarios, except that forward checking costs more per node than plain consistency checking because it filters every remaining domain after each assignment. For these small instances the cost is immaterial; the node counts, which are independent of implementation details, are the more meaningful measure of how much a technique reduces the search.

C. Generated Study Plan

Table IX shows the schedule generated for scenario S2 with MRV and forward checking. Dashes indicate unused or blocked slots. Sessions for Chemistry, whose examination comes first, are placed on the earliest days; Mathematics follows; and subjects with later examinations are placed later, consistent with the examination-deadline constraint H3. No slot holds more than one session (H1), and no day exceeds the load limit (H4).

TABLE IX
GENERATED PLAN FOR SCENARIO S2

Day	Slot 1	Slot 2	Slot 3	Slot 4
1	-	-	Chem	Chem
2	-	Chem	Math	Math
3	Math	Math	-	-
4	-	-	Bio	Bio
5	Bio	Physics	Physics	CS
6	Physics	Physics	CS	English
7	CS	English	English	-

The plan can also be assessed against the soft preferences of Table IV. Every subject's sessions are spread over at least two days (S2), and no day exceeds four sessions (S4). With respect to S3, however, Chemistry and Mathematics have no session on the two days preceding their last study day, so the plan incurs a revision penalty of two even though it is fully valid. This illustrates the division of labour between hard and soft constraints: the search guarantees validity, but the first valid schedule it finds is not necessarily the most useful one. Continuing the search past the first solution and keeping the lowest-cost schedule, as described in Section VIII-D, is the mechanism by which such penalties can be reduced.

	Slot 1	Slot 2	Slot 3	Slot 4
Day 1	-	-	Chem	Chem
Day 2	-	Chem	Math	Math
Day 3	Math	Math	-	-
Day 4	-	-	Bio	Bio
Day 5	Bio	Physics	Physics	CS
Day 6	Physics	Physics	CS	English
Day 7	CS	English	English	-

Fig. 3. Colour-coded plan view for scenario S2 as displayed in the interface.

D. Scalability

To examine how the search behaves as plans grow, the number of subjects was increased from three to ten, with three sessions per subject, a fourteen-day horizon, four slots per day, and fourteen randomly blocked slots (a fixed random seed was used for repeatability). The results are given in Table X.

TABLE X
SCALING WITH NUMBER OF SUBJECTS

Subjects	Variables	MRV nodes	MRV ms	MRV+FC nodes	MRV+FC ms
3	9	45	0.06	9	2.0
4	12	78	0.18	12	3.0
5	15	120	0.16	15	7.0
6	18	171	0.26	18	10.1
7	21	231	0.33	21	14.1
8	24	300	0.43	24	13.3
9	27	378	0.84	27	30.0
10	30	466	0.59	30	33.4

All eight instances were solved without any backtracking. With forward checking, the number of nodes equals the number of variables, meaning that each session was placed on its first attempt; with MRV alone the node count grows more quickly because the search tests candidate values that forward checking would have pruned in advance. The elapsed time for MRV+FC grows with problem size because each assignment triggers a filtering pass over all remaining domains, but remains in the tens of milliseconds for thirty sessions, comfortably within the requirements of an interactive application. These instances are comparatively

loose; the hard cases are those, like S2, in which deadlines and blocked slots leave little slack, and for these the node counts in Table VIII are the better indication of the benefit of the heuristic.

E. Comparison with Existing Approaches

Table XI compares StudySync AI qualitatively with the manual and calendar-based approaches described in Section V.

TABLE XI
QUALITATIVE COMPARISON

Criterion	Manual timetable	Calendar / to-do app	StudySync AI
Builds the plan automatically	No	No	Yes
Guarantees no overlaps	No	Partly	Yes
Respects exam deadlines	By hand	By hand	Yes (H3)
Detects infeasible plans	No	No	Yes
Re-plans on changes	Manual	Manual	Automatic
Explains decisions	No	No	Yes

The comparison is qualitative and reflects the capabilities of the approaches rather than a user study. Calendar applications are valuable for recording and reminding, and StudySync AI is complementary to them: its plan could be exported into a calendar once generated.

F. Discussion

The results support three observations. First, constraint-based search yields schedules that are valid by construction with respect to the hard constraints, which removes an entire class of errors that manual planning invites. Second, MRV is most valuable on problems that are neither trivially easy nor impossible: it made the difference between failing and succeeding within the node budget on S2. Third, the ability to prove infeasibility is as important as the ability to find solutions, since it lets the system give the student actionable feedback.

It should be noted that the scenarios here are small and synthetic, and that the figures describe the behaviour of the reference implementation on those scenarios; they are not benchmarks of a production system. Larger studies with real student data would be needed to measure how the approach performs across diverse timetables and how students perceive the generated plans.

XI. TESTING AND VALIDATION

Testing of StudySync AI is organized around the constraints it is meant to guarantee. The test plan in Table XII lists the checks applied to the generated schedules and the behaviour expected in each case. Because the validator re-checks the hard constraints independently of the search, every generated schedule is verified before it is displayed.

TABLE XII
TEST PLAN

No.	Test condition	Expected behaviour
T1	Typical input with feasible requirements	A valid plan is displayed
T2	Two sessions competing for one slot	Only one is placed; the other moves
T3	Session would fall after its exam	Placement rejected (H3)
T4	Slot marked unavailable	Slot never used (H2)
T5	Daily load limit reached	No further sessions that day (H4)
T6	Requirements exceed available slots	Infeasibility reported, no plan
T7	Same input run twice	Identical plan (deterministic ties)
T8	Invalid exam date entered	Input rejected with a message

Unit tests target individual modules—the model builder is checked for correct domains, the search engine for completeness on small instances whose solutions are known, and the validator for detecting each type of violation when given hand-constructed invalid schedules. Interface testing confirms that each of the four views displays the information derived from the current inputs and updates when inputs change.

A. Validation of Generated Schedules

Beyond the individual test cases, every schedule produced by the system is checked by a post-condition: the validator iterates over all pairs of sessions to confirm that no two share a slot, over all sessions to confirm that each lies in an available slot before its examination, and over all days to confirm the load limits. Because this check is independent of the search, it also serves as an oracle for testing—randomly generated inputs can be run through the planner and the output validated automatically, and any failure indicates a defect in either the model builder or the search engine.

B. Planned Usability Evaluation

The technical tests establish that schedules are valid; they do not establish that students find them useful. A planned usability evaluation would ask a small group of students to prepare a real examination timetable using the system and compare it with their usual method, collecting measures such as the time taken to produce a plan, the number of manual adjustments required, and the student's rating of the plan's balance across subjects. Such an evaluation was beyond the scope of the present prototype and is listed under future work.

XII. LIMITATIONS AND FUTURE SCOPE

A. Limitations

- Sessions are of uniform length; subjects requiring longer uninterrupted blocks are modelled as several consecutive sessions.
- The system assumes that the student's availability and the number of sessions required are supplied

accurately; it does not estimate how long a topic will take.

- Soft-preference weights are fixed rather than learned from the student's behaviour.
- The search finds a valid schedule and can optimize among those found, but does not guarantee the globally optimal plan for large instances.
- Plans are static once generated; they do not automatically adapt to missed sessions.

B. Future Scope

- Add arc-consistency (AC-3) preprocessing to prune domains before search begins [2].
- Incorporate local search or branch-and-bound to improve soft-preference cost on larger problems [9].
- Support automatic re-planning when a session is missed, preserving as much of the existing plan as possible.
- Integrate with calendar services to import existing commitments as unavailable slots.
- Allow spaced-repetition patterns as explicit constraints, building on distributed-practice findings [13], [14].
- Offer export of the plan to common calendar and document formats.

C. Usability and Responsible Use

A study planner shapes how students spend their time, so it should support rather than constrain them. The generated plan is a proposal that the student remains free to modify, and the interface makes the underlying constraints visible so that students can judge whether the plan reflects what they want. The system stores only the information needed to build the schedule—subject names, dates, and availability—and does not require personal identifiers. Since the planner optimizes for coverage and timing, it cannot judge how well a student is actually learning; it should be used alongside the student's own judgement and, where appropriate, the guidance of teachers.

D. Threats to Validity

The experimental results should be interpreted with caution. The scenarios are synthetic and small, the measure of search effort depends on the particular constraint set and tie-breaking rules, and the behaviour of MRV is known to vary between problem instances: it provides large savings on tightly constrained problems and little on loose ones, as the comparison of S1 and S2 shows. The reported times come from a single unoptimized implementation. The findings therefore illustrate the behaviour of the technique rather than establishing performance bounds.

XIII. LEARNING OUTCOMES

Through this project the following concepts were understood and applied:

- Formulating a real-world problem as a Constraint Satisfaction Problem with variables, domains, and constraints.
- Distinguishing hard constraints from soft preferences and expressing both formally.
- Implementing backtracking search and understanding its completeness.
- Applying the MRV heuristic and forward checking and measuring their effect on search effort.
- Detecting conflicts and proving infeasibility through exhaustive, pruned search.
- Separating search from validation to improve correctness.
- Designing a modular Python application and a Streamlit interface.
- Presenting AI decision-making in an explainable manner.
- Appreciating the role of classical AI methods where machine learning is unnecessary.

XIV. CONCLUSION

StudySync AI demonstrates that an everyday academic problem—arranging study sessions around examination dates, available hours, and subject priorities—can be solved effectively with classical AI. By modelling study sessions as variables, time slots as domains, and academic requirements as constraints, the system turns a vague planning task into a precisely defined CSP. Backtracking search with the MRV heuristic and forward checking finds conflict-free schedules efficiently, the validation mechanism guarantees that hard constraints are respected, and soft preferences guide the system toward plans that promote balanced and well-timed preparation.

The experiments on the reference implementation show that MRV and forward checking reduce the search effort substantially on moderately constrained problems and that the system can identify infeasible requirements instead of producing an invalid plan. Implemented in Python with a Streamlit interface, the system presents inputs, constraints, the decision process, and the final plan clearly, and does so without machine learning or deep learning, so that every decision remains explainable. Ultimately, StudySync AI aims to improve study organization, promote balanced preparation across subjects, reduce scheduling conflicts, and support students in developing consistent and effective study habits.

Acknowledgment

The authors thank the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for the guidance and facilities provided during the course Foundations of Artificial Intelligence.

References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] A. K. Mackworth, "Consistency in networks of relations," Artif. Intell., vol. 8, no. 1, pp. 99–118, 1977.

[3] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," Artif. Intell., vol. 14, no. 3, pp. 263–313, 1980.

[4] R. Dechter, Constraint Processing. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[5] V. Kumar, "Algorithms for constraint-satisfaction problems: A survey," AI Mag., vol. 13, no. 1, pp. 32–44, 1992.

[6] E. Tsang, Foundations of Constraint Satisfaction. London, U.K.: Academic Press, 1993.

[7] F. Rossi, P. van Beek, and T. Walsh, Eds., Handbook of Constraint Programming. Amsterdam, The Netherlands: Elsevier, 2006.

[8] A. Schaerf, "A survey of automated timetabling," Artif. Intell. Rev., vol. 13, no. 2, pp. 87–127, 1999.

[9] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," Eur. J. Oper. Res., vol. 140, no. 2, pp. 266–280, 2002.

[10] J. R. Bitner and E. M. Reingold, "Backtrack programming techniques," Commun. ACM, vol. 18, no. 11, pp. 651–656, 1975.

[11] S. W. Golomb and L. D. Baumert, "Backtrack programming," J. ACM, vol. 12, no. 4, pp. 516–524, 1965.

[12] S. C. Brailsford, C. N. Potts, and B. M. Smith, "Constraint satisfaction problems: Algorithms and applications," Eur. J. Oper. Res., vol. 119, no. 3, pp. 557–581, 1999.

[13] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, "Distributed practice in verbal recall tasks: A review and quantitative synthesis," Psychol. Bull., vol. 132, no. 3, pp. 354–380, 2006.

[14] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving students' learning with effective learning techniques," Psychol. Sci. Public Interest, vol. 14, no. 1, pp. 4–58, 2013.

[15] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io